

**A DISSONÂNCIA ESTRUTURAL ENTRE O PERFIL
PROFISSIONAL EXIGIDO PELO MERCADO DE TECNOLOGIA
DA INFORMAÇÃO E A FORMAÇÃO FORMAL:
uma pesquisa bibliográfica à luz da sociologia do trabalho e da
engenharia de software**

RESUMO

O presente artigo investiga a dissonância entre a formação acadêmica em Engenharia de Software estruturada em modelos tradicionais "Cascata" e a adoção mercadológica de qualificações aceleradas de características "Ágeis". Diante da obsolescência tecnológica, cursos utilitaristas prometem valor imediato, mas formam executores técnicos com baixo raciocínio crítico, incapazes de questionar a lógica estrutural dos sistemas. O estudo diferencia o papel do Engenheiro de Software, pautado por métricas de governança e ética, do mero desenvolvedor de ferramentas. Analisa-se como a lacuna na pedagogia tecnológica de base, aliada ao uso acrítico da Inteligência Artificial, amplifica a produção de códigos frágeis, flertando com uma nova "crise do software". Evocando catástrofes históricas como o caso Therac-25, argumenta-se que a negligência arquitetural e a delegação de decisões à IA geram riscos jurídicos e físicos irreversíveis para a sociedade. Conclui-se que a superação desse cenário exige a valorização do conhecimento estruturado e a rigorosa responsabilização legal, resguardando a infraestrutura digital contra o pragmatismo cego.

Palavras-chave: Engenharia de Software. Ética Profissional. Inteligência Artificial. Responsabilidade Jurídica. Obsolescência Tecnológica.

1 – INTRODUÇÃO

A consolidação da Tecnologia da Informação (TI) como a infraestrutura nevrálgica da sociedade contemporânea transformou o software em uma entidade ciberfísica onipresente, responsável por orquestrar desde o núcleo de transações do sistema financeiro global até os protocolos de suporte vital em ambientes hospitalares. Neste cenário de dependência absoluta, este artigo tem como objetivo

geral analisar criticamente a dissonância estrutural entre o perfil profissional utilitarista exigido pelo mercado e a formação acadêmica formal em Engenharia de Software. Observa-se um grave descompasso epistemológico: enquanto a academia tradicional opera na cadência de um amadurecimento lógico e arquitetural de longo prazo frequentemente assemelhado à inércia do modelo "Cascata", o mercado clama por agilidade extrema. Essa urgência fomentou a proliferação de cursos intensivos e *bootcamps* que prometem rápida inserção profissional, mas que acabam por instituir uma perigosa "pedagogia da superficialidade". O resultado é uma redução epistemológica da engenharia, criando um abismo conceitual entre o mero "desenvolvedor" treinado para operar *frameworks* de forma efêmera e o verdadeiro "Engenheiro de Software", cuja atuação pressupõe rigor em arquiteturas limpas, *compliance* normativo, tolerância a falhas e comportamento ético.

Essa crise formativa tem raízes na própria pedagogia tecnológica de base. O mercado absorve gerações de jovens frequentemente rotulados como "nativos digitais", mas que, na realidade, possuem letramento apenas para o consumo de interfaces, carecendo de profundidade analítica para questionar o processamento lógico das máquinas. Quando essa mão de obra desprovida de alicerces matemáticos e algorítmicos se depara com a introdução massiva da Inteligência Artificial (IA) generativa, o cenário agrava-se drasticamente. O que deveria ser um copiloto de produtividade converte-se em uma muleta cognitiva. Ao delegar o pensamento arquitetural à IA, consolida-se a "automação da ignorância", onde profissionais produzem sistemas "caixa-preta" que eles próprios são incapazes de auditar ou refatorar.

A consequência corporativa dessa dinâmica é a chamada "inflação de senioridade". Diante da escassez de talentos, o mercado promove precocemente operadores de código a cargos estratégicos de arquitetura. Sem a vivência em engenharia de sistemas e carecendo de visão de ciber-resiliência, essas lideranças artificiais tomam decisões estruturais que culminam em dívidas técnicas impagáveis. Diante desse panorama, levanta-se a seguinte problematização: em que medida a substituição da formação analítica rigorosa por treinamentos de execução superficial, somada à automação da ignorância via IA e à inflação de

senioridade, enfraquece a responsabilidade ética do profissional e expõe a infraestrutura sociotécnica a *bugs* críticos e passivos jurídicos irreversíveis?

Para investigar e responder a essa questão, a presente pesquisa caracteriza-se como qualitativa, de tipologia teórica e bibliográfica, fundamentando-se em obras da sociologia do trabalho, da engenharia de software e do direito digital. A abordagem metodológica adotada é a crítico-dialética, confrontando a lógica produtivista com a necessidade imperativa de qualidade sistêmica.

A justificativa para este estudo pauta-se na urgência de combater o pragmatismo cego na construção de sistemas de missão crítica. A negligência arquitetural e as falhas de operações humanas em cima da TI deixaram de causar apenas ineficiências virtuais para gerar colapsos reais. Vazamentos massivos de dados que ferem frontalmente a Lei Geral de Proteção de Dados (LGPD), falhas em integrações que paralisam sistemas bancários inteiros e catástrofes físicas irremediáveis como o emblemático caso da máquina de radioterapia Therac-25 são sintomas da mesma doença: a precarização técnica. Discutir a responsabilidade objetiva, cível e criminal de quem arquiteta a tecnologia tornou-se o único mecanismo viável para garantir que o software não atue como um vetor de risco, mas como um escudo de governança e sustentabilidade para a sociedade digital.

2 – DESENVOLVIMENTO

2.1 – ANÁLISE DE REQUISITOS E A MATRIZ PEDAGÓGICA (O PLANEJAMENTO FALHO)

Na engenharia de software canônica, a fase de Análise de Requisitos estabelece as fundações arquiteturais, os limites operacionais e a viabilidade lógica de um sistema. Qualquer vício ou omissão inserido nesta etapa primordial reverbera catastroficamente por todo o Ciclo de Vida de Desenvolvimento de Software (SDLC). Ao transpormos essa premissa metodológica para a sociologia do trabalho em Tecnologia da Informação (TI), compreende-se que a "Fase de Requisitos" do profissional reside na sua matriz pedagógica e educacional base. Contudo, o que se observa na contemporaneidade é um colapso formativo profundo, caracterizado por requisitos cognitivos mal estabelecidos, agravado pela

tensão latente entre a urgência utilitarista do mercado e o rigor da educação científica formal.

Para compreender essa falha de "planejamento" na formação da mão de obra, é imperativo analisar a ontologia da TI sob a ótica da "Modernidade Líquida", do sociólogo Zygmunt Bauman. A transição ininterrupta para uma sociedade hiperconectada consolidou a infraestrutura digital como o alicerce absoluto das relações socioeconômicas modernas. Diferentemente das ciências tradicionais, a TI atua sobre construtos artificiais, impulsionados pela evolução natural e inexorável do ecossistema de tecnologia global. O conhecimento técnico abdica do seu caráter vitalício para assumir a forma fria de mercadoria perecível, sujeita a uma obsolescência exógena avassaladora. O modelo tradicional de ensino superior evidencia as suas fraturas ao tentar combater essa liquidez operando sob a égide do que pode ser denominado "Waterfall educacional". Assim como o modelo em cascata na engenharia é linear e inflexível, as diretrizes curriculares acadêmicas são projetadas de forma estática para um futuro de longo prazo, tornando-se técnica e comercialmente obsoletas meses depois.

O agravante sistêmico dessa fase inicial falha é o mito dos "nativos digitais". As novas gerações ingressam no funil de formação tecnológica com um vasto letramento para o consumo de interfaces abstraídas, mas com um severo déficit de raciocínio estrutural. Tratam os computadores como entidades herméticas, carecendo da capacidade analítica para formular os "requisitos" lógicos do pensamento computacional que interligam as camadas de software ao hardware subjacente. Sem o entendimento de como a alocação de memória, o processamento de baixo nível e a integração física operam, o indivíduo é incapaz de prever gargalos de performance ou estruturar defesas contra vulnerabilidades em nível de sistema.

Diante da inércia acadêmica e da escassez global crônica de talentos em tecnologia, o mercado fomentou uma rota de fuga: a explosão na oferta de *bootcamps* cursos de imersão intensiva focados exclusivamente no ensino rápido e prático de linguagens e *frameworks* da moda. O intuito utilitarista é preencher cadeiras operacionais rapidamente para tarefas de codificação repetitiva. No entanto, esses atalhos formativos frequentemente negligenciam por completo a

profundidade teórica, a ética da informação e os fundamentos da ciência. Institui-se uma perigosa "pedagogia da superficialidade", onde o ensino rigoroso é brutalmente suprimido em favor da replicação de tutoriais.

O resultado desta supressão epistemológica é a aniquilação do pensamento arquitetural. O aluno é adestrado na sintaxe de um *framework* específico, mas ignora por completo os paradigmas que o sustentam. Conceitos vitais para a resiliência corporativa como a separação de responsabilidades da *Clean Architecture*, o desacoplamento de infraestrutura promovido pela Arquitetura Hexagonal, ou a orquestração distribuída em microsserviços e SOA (*Service-Oriented Architecture*) são varridos do currículo em nome da velocidade de entrega. Desenvolvedores formados exclusivamente sob essa lógica atuam como meros montadores de componentes, limitando-se a acoplar bibliotecas pré-fabricadas sem a menor compreensão de seu impacto estrutural.

Sob o escrutínio sociológico, essa exigência cruel de atualização perpétua sobre bases frágeis revela dinâmicas de precarização acentuada. O profissional, desprovido dos alicerces lógicos que o tornariam resiliente às mudanças de ferramentas, submete-se àquilo que Byung-Chul Han descreve como a sociedade do cansaço e do desempenho. A tecnologia e os algoritmos de controle são utilizados para intensificar a exploração da força de trabalho, exigindo uma hiper-produtividade insustentável. Ao suprimir a "fase de requisitos" analíticos da mente do engenheiro, o mercado forma uma legião de executores exaustos e obsoletos por *design*, transferindo passivos imprevisíveis para a sociedade civil e pavimentando o terreno para o esgotamento dos recursos nas fases subsequentes do ciclo de vida sistêmico.

2.2 – DESIGN ARQUITETURAL E A INSUSTENTABILIDADE ECOLÓGICA (O PROJETO TÓXICO)

Na engenharia de software canônica, a fase de *Design* Arquitetural é o momento crítico em que os requisitos abstratos são traduzidos em plantas estruturais, definindo os padrões de projeto, a topologia de infraestrutura e as restrições de segurança. É a fase onde a governança e a resiliência de longo prazo devem prevalecer sobre a conveniência imediata. Contudo, na anatomia do nosso

colapso sistêmico, esta etapa é fatalmente corrompida pelo produto direto da fase anterior: a alocação de profissionais de formação superficial e utilitarista em posições de decisão estratégica, um fenômeno amplamente reconhecido no setor como "inflação de senioridade".

Diante do apagão crônico de talentos e da altíssima rotatividade (*turnover*), o mercado de tecnologia subverteu a esteira natural de amadurecimento profissional. Indivíduos com exíguo tempo de atuação clínica em engenharia, frequentemente detentores de meros 2 a 3 anos de experiência em codificação de *front-end* ou *frameworks* específicos, são sumariamente promovidos a "Arquitetos de Software" ou "Tech Leads". Estes "arquitetos artificiais", desprovidos do lastro acadêmico em estruturas de dados, sistemas operacionais e redes de computadores, assumem a prancheta de projetos críticos. O resultado é a tomada de decisões arquiteturais baseadas não na adequação estrutural, mas no *hype* tecnológico momentâneo resultando na adoção precipitada e caótica de arquiteturas de microsserviços que rapidamente degeneram em ineficientes "monolitos distribuídos", multiplicando exponencialmente os pontos de falha da rede.

A primeira vítima letal dessa inépcia no *design* é a abdicação dos princípios de *Security by Design* (Segurança desde a Concepção) e *Privacy by Design* (Privacidade desde a Concepção). Na engenharia estruturada, a criptografia, o controle de acesso e a mitigação de vetores de ataque são alicerces topológicos. Contudo, o arquiteto utilitarista, pressionado pela entrega de valor imediato, relega a segurança a um mero "requisito não-funcional" a ser tratado de forma reativa nas fases finais. Simultaneamente, a ignorância sobre arquiteturas agnósticas (como a Arquitetura Hexagonal ou *Clean Architecture*) conduz as corporações à severa armadilha do *Vendor Lock-in* (aprisionamento tecnológico). Ao invés de projetar sistemas independentes, o *design* acopla-se viciosamente a serviços gerenciados proprietários de *Big Techs* (AWS, Google Cloud, Azure). A arquitetura deixa de ser nativa da nuvem (*Cloud Native*) para se tornar cativa da nuvem (*Cloud Captive*), resultando em faturas infraestruturais exorbitantes e perda total de soberania sistêmica.

Além da asfixia financeira e dos riscos de vazamento de dados (LGPD), a consequência material dessa inépcia é o fenômeno do "Peso do Código" (*Code Bloat*). Sem a capacidade crítica para avaliar a complexidade ciclomática ou a eficiência assintótica de seus algoritmos (ignorando a diferença de custo computacional entre uma solução linear e algoritmos de força bruta que operam em $O(n^2)$), esses profissionais recorrem ao acoplamento indiscriminado de bibliotecas pré-fabricadas. Sistemas que outrora exigiam megabytes de memória passam a exigir gigabytes, mascarando a degradação da engenharia humana sob o manto da evolução do hardware (a falaciosa Lei de Wirth).

A gravidade do *Code Bloat*, contudo, transcende a ineficiência virtual e adentra o domínio da emergência climática global. A negligência na fase de *design* arquitetural configura uma externalidade ambiental devastadora, chocando-se frontalmente com a urgência da Sustentabilidade Computacional e da *Green Software Engineering* (TI Verde). Em escala de *Cloud Computing*, operando em alta disponibilidade para milhões de usuários, cada ciclo extra de processamento (CPU) e de memória (RAM) desperdiçado por um código ineficiente traduz-se num consumo colossal de energia elétrica e num volume estrondoso de água industrial desviada para os sistemas de resfriamento dos *Data Centers*.

Esta pegada ecológica atinge proporções catastróficas quando a inflação de senioridade cruza com o uso irresponsável da Inteligência Artificial Generativa no *design*. Quando um arquiteto júnior delega à IA a elaboração de lógicas sistêmicas sem curadoria humana, ele incita um consumo duplo: gasta-se energia massiva consultando a IA para gerar um código sujo, e gasta-se energia ininterrupta nos servidores globais para executar essa arquitetura imperfeita. O *design* mal projetado deixa, portanto, de ser apenas uma dívida técnica corporativa para se materializar como um projeto tóxico, operando como um ralo ambiental que drena os recursos físicos do planeta sob a égide do "pragmatismo ágil".

2.3 – IMPLEMENTAÇÃO ESTOCÁSTICA: DA IA GENERATIVA À AGI (A CODIFICAÇÃO TERCEIRIZADA)

Na taxonomia clássica do Ciclo de Vida de Desenvolvimento de Software (SDLC), a fase de Implementação é a etapa material onde os diagramas

arquiteturais são traduzidos em linhas de código executável. Historicamente, a engenharia de software foi pautada por lógicas sistêmicas inerentemente determinísticas e rígidas. Em sistemas tradicionais, a previsibilidade era a regra de ouro: uma anomalia em uma transação comercial decorria de um erro lógico linear e rastreável, o que permitia auditorias diretas e correções cirúrgicas. Contudo, a integração profunda da Inteligência Artificial (IA), com ênfase nos Grandes Modelos de Linguagem (LLMs) e na IA Generativa (IAG), introduziu um paradigma estocástico nos processos de desenvolvimento deste século. Essa transição brutal de um ambiente rígido e controlável para um ecossistema heurístico e probabilístico representa a força mais disruptiva e perigosa na disciplina da engenharia contemporânea.

A premissa mercadológica vende os modelos generativos como assistentes de codificação capazes de aliviar a carga cognitiva ao automatizar rotinas repetitivas. No entanto, quando esta tecnologia é inserida num ecossistema já precarizado por profissionais formados sob a "pedagogia da superficialidade" a qual suprimiu o raciocínio matemático base, consolida-se um fenômeno devastador: a "automação da ignorância". Desenvolvedores sem base profunda em algoritmos tornam-se reféns orgânicos dessas ferramentas. O programador abdica do raciocínio estrutural e delega o pensamento lógico à máquina, interagindo com o sistema que ele próprio constrói como se fosse uma insondável "caixa-preta". O impacto é assustadoramente quantificável: estudos avaliando bases como o DevGPT evidenciaram que trechos de código gerados por LLMs apresentam uma estimativa alarmante de, em média, 20,49 potenciais *bugs* por *snippet* gerado.

Esse cenário de terceirização do pensamento deflagra um segundo fenômeno insidioso: a inversão do ônus cognitivo. Na engenharia de software canônica, a máxima de que "ler código é mais difícil do que escrevê-lo" é universalmente aceita. Ao utilizar a IA, o desenvolvedor júnior gera centenas de linhas de código em segundos, mas transfere o massivo ônus de auditoria, depuração e manutenção dessa lógica espaguete e estocástica para os engenheiros seniores. O ganho ilusório de produtividade individual na escrita converte-se em um gargalo paralisante na fase de revisão (*Code Review*), inflando

os custos operacionais de longo prazo e deteriorando a manutenibilidade do repositório.

Ainda mais crítico é o risco inerente à homogeneização da base de código global. A adoção massiva das mesmas ferramentas de IA (como Copilot ou ChatGPT), treinadas invariavelmente sobre os mesmos repositórios públicos (como o GitHub), cria o que se pode conceituar como uma "monocultura de software". Traçando um paralelo com a biologia, onde uma monocultura agrícola é devastada por um único patógeno devido à ausência de diversidade genética, a TI contemporânea caminha para um abismo análogo. Milhões de desenvolvedores rasos estão replicando a mesma estrutura de código gerada pela máquina em sistemas corporativos distintos. A descoberta de uma única vulnerabilidade (*Zero-day exploit*) neste padrão algorítmico globalizado possui o potencial de comprometer, simultaneamente, milhares de infraestruturas ao redor do globo, elevando falhas locais ao patamar de riscos sistêmicos de segurança nacional.

A esta vulnerabilidade topológica soma-se o "veneno jurídico" da contaminação de Propriedade Intelectual (IP). A IA generativa foi treinada absorvendo bilhões de linhas de código, muitas das quais protegidas por direitos autorais e licenças restritivas (como a GPL, de caráter "viral"). Quando o "montador de componentes" utilitarista solicita um bloco de código à IA e o injeta no núcleo de um sistema bancário ou de um SaaS proprietário, ele pode estar inadvertidamente incorporando código de terceiros sujeito a *copyright*. Essa prática leviana escancara corporações a processos multibilionários de quebra de patente e violação de licenças, provando que o código gerado por IA sem curadoria é um passivo tanto técnico quanto jurídico.

A gravidade destas camadas de risco assume contornos de ameaça existencial quando projetamos o futuro imediato da tecnologia em direção à AGI (Inteligência Artificial Geral). Se a indústria tecnológica contemporânea já demonstra inépcia formativa para auditar o ônus cognitivo, a monocultura e a contaminação de IP gerados por IAs restritas, a perspectiva de delegar o *design* de sistemas de missão crítica a uma AGI por profissionais desqualificados flerta com o colapso infraestrutural absoluto. Especialistas alertam enfaticamente que, sem a revisão rigorosa por parte de Engenheiros de Software humanos em um inegociável

modelo *human-in-the-loop*, a inserção contínua destes códigos estocásticos em esteiras de liberação ágil (CI/CD) pavimenta uma via expressa para catástrofes incalculáveis, corrompendo a sustentabilidade da infraestrutura civil global.

2.4 – TESTES, DÍVIDA TÉCNICA E O SALTO QUÂNTICO (A VULNERABILIDADE EXTREMA)

Na taxonomia do Ciclo de Vida de Desenvolvimento de Software (SDLC), a fase de Testes e Homologação (*Quality Assurance* - QA) atua como o último bastião de sanidade arquitetural antes que um artefato digital interaja com o mundo real. É a fronteira metodológica desenhada para validar requisitos funcionais, estressar limites de alocação de memória e garantir a blindagem contra vetores de ataque cibernético. No entanto, sob a ditadura do "ágil deturpado" e a incessante pressão mercadológica por entregas contínuas, esta fase foi sistematicamente aniquilada. A cultura corporativa, muitas vezes balizada pelo pragmatismo tóxico do *move fast and break things* (mova-se rápido e quebre as coisas), subverteu o rigor analítico. Práticas indispensáveis de engenharia, como o Desenvolvimento Orientado a Testes (TDD), a cobertura massiva de testes unitários e as auditorias profundas de segurança (*Pentests* e cultura *DevSecOps*), passaram a ser tratadas por gestores leigos como gargalos operacionais inaceitáveis. O escopo de testes foi antieticamente transferido para a fase de produção, transformando a sociedade civil em *beta testers* compulsórios de infraestruturas de missão crítica.

Esse cenário de negligência é exponencialmente agravado por um novo fenômeno decorrente da automação irrestrita: a "câmara de eco algorítmica". Na tentativa de simular métricas de qualidade (como a cobertura de código exigida por esteiras de CI/CD), desenvolvedores de formação superficial recorrem à mesma Inteligência Artificial Generativa que codificou a aplicação para redigir os seus testes unitários e de integração. Institui-se, assim, um viés de confirmação automatizado e um perigoso loop de retroalimentação: a máquina testa a máquina. Um *bug* lógico, um viés estocástico ou uma falha de segurança introduzida pela IA na fase de implementação será, por consequência arquitetural, validado e chancelado pelo teste concebido pela própria IA. O Engenheiro de Software humano, ao abdicar do papel de auditor independente, chancelará relatórios de

qualidade vazios e falaciosos, criando uma falsa sensação de segurança perante *code smells* e falhas endêmicas.

O subproduto financeiro e estrutural dessa negligência sequencial que começa na falta de base pedagógica, passa por um *design* tóxico e culmina numa codificação estocástica cegamente testada por algoritmos é a transmutação da Dívida Técnica em "Falência Técnica" (*Technical Bankruptcy*). O que Ward Cunningham originariamente concebeu como uma metáfora para descrever concessões arquiteturais conscientes feitas a favor de um ganho temporal estratégico (dívida esta que deveria ser paga com "juros" de refatoração futura), tornou-se um buraco negro orçamentário e incontrolável. Sistemas corporativos tornam-se vastos repositórios de código legado incompreensível (*spaghetti code*). O custo cognitivo e operacional de manutenção atinge o ponto de ruptura: a empresa sofre a "falência técnica", ficando refém do seu próprio produto defeituoso, incapaz de adicionar novas funcionalidades sem desencadear cascatas de erros imprevisíveis.

Contudo, a verdadeira magnitude dessa vulnerabilidade extrema será exposta de forma letal e irreversível por um evento de extinção em massa para a TI tradicional: o salto para a Computação Quântica. A iminência do chamado "Q-Day" (o momento em que computadores quânticos criptograficamente relevantes se tornarem plenamente operacionais) representa a maior ameaça à segurança da informação da história moderna. O Algoritmo de Shor, executado de forma polimorfa em um processador quântico de alta fidelidade, possui a capacidade matemática comprovada de fatorar números primos gigantescos em tempo polinomial, estilhaçando instantaneamente os pilares da criptografia assimétrica atual (como o RSA e a Criptografia de Curvas Elípticas - ECC). Num horizonte temporal curtíssimo, todo o sigilo bancário, comunicações militares, prontuários de saúde (submetidos à LGPD) e protocolos globais de internet estarão completamente indefesos.

A mitigação deste apocalipse cibernético exige uma transição monumental e global para a Criptografia Pós-Quântica (PQC). É exatamente neste vértice de crise civilizatória que a dissonância estrutural na formação dos profissionais de tecnologia cobrará o seu preço definitivo. A implementação de

algoritmos PQC não se resume a atualizar dependências em uma biblioteca de pacotes; ela baseia-se em problemas matemáticos de altíssima complexidade (como a criptografia baseada em reticulados - *Lattice-based cryptography*). Exige-se reescrever a fundação lógica de sistemas legados que já se encontram em estado de falência técnica.

E é aqui que a "pedagogia da superficialidade" evidencia o seu colapso. O mercado, ao suprimir o ensino rigoroso de álgebra linear, matemática discreta e complexidade algorítmica em favor do treinamento utilitarista, formou uma força de trabalho estruturalmente incapaz de compreender e implementar a matemática de sobrevivência da era quântica. O profissional de qualificação acelerada, adestrado para atuar como um mero usuário de *frameworks*, não possui o lastro epistemológico necessário para salvar a infraestrutura corporativa da obsolescência criptográfica. A falência da fase de testes humanos e o acúmulo irresponsável de código lixo gerado por IA criaram um labirinto entrópico irreversível; a escassez de Engenheiros de Software de base sólida garante que não haverá resiliência intelectual suficiente para refatorar e blindar a malha digital antes que o colapso quântico exponha o tecido socioeconômico global.

2.5 – IMPLANTAÇÃO, MANUTENÇÃO E A GUILHOTINA JURÍDICA (O RISCO NO MUNDO REAL)

No ápice do Ciclo de Vida de Desenvolvimento de Software (SDLC), a fase de Implantação (*Deploy*) e Manutenção representa o instante crítico em que o código abstrato transborda para a malha sociotécnica global, passando a orquestrar infraestruturas físicas, transações financeiras e, em última instância, vidas humanas. É nesta fronteira irreversível que a expansão visceral da tecnologia impõe uma distinção estrutural, ética e inadiável entre o mero "desenvolvedor" e o verdadeiro "Engenheiro de Software". Enquanto o primeiro atua como um operador focado na entrega rápida de funcionalidades jogando o código "por cima do muro" para o ambiente de produção, o Engenheiro pauta-se por métricas de governança, práticas de Engenharia de Confiabilidade de Sites (SRE) e comportamento ético inegociável. A engenharia exige a assunção de responsabilidades técnicas, materializando-se em obrigações formais, como a legitimação jurídica por meio de

conselhos de classe (Sistema CONFEA/CREA) e a emissão obrigatória de Anotações de Responsabilidade Técnica (ART) para atestar a segurança de infraestruturas cibernéticas.

Quando o pragmatismo cego, a inflação de senioridade e a pressão mercantil prevalecem sobre o rigor arquitetural e a homologação exaustiva, a implantação de um software converte-se em um gatilho para tragédias no mundo físico. O caso da máquina de radioterapia Therac-25, na década de 1980, assombra a engenharia como o fantasma definitivo da negligência metodológica no *deploy*: a confiança absoluta em um código não testado, aliada à remoção de travas físicas, resultou na administração autônoma de doses letais de radiação. Na contemporaneidade, a delegação de decisões sistêmicas a códigos gerados acriticamente e implantados sob a pressão do "ágil deturpado" replica, em escala logarítmica, a arrogância estrutural do Therac-25.

Para além dos danos diretos, a imperícia técnica gera colapsos planetários. O recente e catastrófico apagão da CrowdStrike (2024) atua com notório peso empírico: o *deploy* precipitado de uma configuração falha no nível do *kernel*, sem os devidos testes gradativos (*canary deployments*), congelou a logística de portos, paralisou hospitais em metrópoles e impediu frotas inteiras de aviões de decolar. Contudo, o colapso na fase de implantação divide espaço com uma vulnerabilidade ainda mais silenciosa na fase de Manutenção: os ataques à Cadeia de Suprimentos de Software (*Supply Chain Attacks*).

A engenharia contemporânea baseia-se pesadamente na importação de bibliotecas de código aberto. O desenvolvedor utilitarista acopla esses pacotes, mas a corporação falha criminalmente em mantê-los auditados. Quando vulnerabilidades críticas de dia zero (*Zero-days*) são descobertas em pacotes onipresentes como o infame caso do *Log4j*, a ausência de uma cultura de *DevSecOps* e de ferramentas de observabilidade e telemetria deixa as organizações voando às cegas. Sem um inventário de software (SBOM - *Software Bill of Materials*) rigoroso, as corporações são incapazes de rastrear onde o código vulnerável está rodando, permitindo que invasores sequestram servidores globais e exfiltrem terabytes de dados antes mesmo que a equipe de TI compreenda a extensão do ataque.

A consequência desta inépcia na manutenção choca-se frontalmente com a Lei Geral de Proteção de Dados Pessoais (LGPD) e normativas internacionais. O vazamento em massa de informações privadas ocasionado por falhas rudimentares como a não atualização de bibliotecas vulneráveis (CVEs) ou o esquecimento de desativar o acesso público em *buckets* de armazenamento em nuvem acarreta na imposição de multas governamentais exorbitantes. Diante deste potencial de dano, o ordenamento jurídico rompeu com a complacência histórica da indústria. A prática de corporações que buscam se isentar por meio de Acordos de Licença (EULAs), comercializando sistemas críticos sob a irresponsável cláusula "as is" (no estado em que se encontra), tornou-se moralmente inaceitável e juridicamente nula. O Código de Defesa do Consumidor (CDC) e o Código Civil brasileiro consagram a responsabilização puramente objetiva e solidária de toda a cadeia de provisão de tecnologia.

Ainda mais alarmante para os executores utilitaristas é a incidência do Direito Penal e o fenômeno do "bode expiatório" corporativo. A implantação precipitada de sistemas falhos em infraestruturas de missão crítica configura crime nas modalidades estritas de imperícia, imprudência ou negligência, tipificadas no Art. 18 do Código Penal Brasileiro. Quando a "Guilhotina Jurídica" desce após um desastre cibernético, as corporações frequentemente tentam blindar suas diretorias transferindo a culpa criminal para a base operacional (o desenvolvedor júnior que executou o *script*). É neste exato momento de crise que a verdadeira Engenharia de Software demonstra seu valor absoluto: a conformidade digital (*compliance*), a arquitetura rastreável e a emissão documentada de ARTs deixam de ser meros diferenciais de currículo para se erguerem como o único escudo forense viável. Assinar a responsabilidade técnica obriga a corporação a respeitar os vetos de segurança do Engenheiro, garantindo não apenas a integridade da infraestrutura digital da sociedade, mas a defesa legal intransigível do próprio profissional frente à barbárie do mercado utilitarista.

3 – CONCLUSÃO

A análise crítica conduzida neste estudo evidencia que a dissonância estrutural entre a formação acadêmica formal e as exigências imediatistas do

mercado de Tecnologia da Informação configura uma crise sistêmica de proporções civilizatórias. Respondendo de forma categórica à problemática inicial, conclui-se que a substituição de uma matriz epistemológica rigorosa por modelos de qualificação acelerada e utilitarista não apenas precariza o perfil profissional, mas enfraquece mortalmente a responsabilidade ética, expondo a infraestrutura sociotécnica global a falhas catastróficas e passivos jurídicos irreversíveis.

Ao dissecar essa crise através das fases do Ciclo de Vida de Desenvolvimento de Software (SDLC), restou comprovado que o colapso é cumulativo. A inércia do modelo educacional e a adoção da "pedagogia da superficialidade" na Fase de Requisitos formam profissionais desprovidos de base matemática e arquitetural. Essa lacuna desdobra-se em projetos tóxicos e insustentáveis, onde a inflação de senioridade impulsiona o aprisionamento tecnológico (*vendor lock-in*), a negligência com o *Privacy by Design* e o fenômeno do *Code Bloat*, drenando os recursos físicos do planeta. Na fase de Implementação, a delegação do pensamento lógico à Inteligência Artificial consolida a monocultura de software e a "automação da ignorância", injetando códigos estocásticos que preparam um terreno sombrio para a iminência de uma AGI.

O abandono do rigor analítico culmina na aniquilação da fase de Testes, onde a "câmara de eco algorítmica" acelera a falência técnica das corporações. Como demonstrado, essa fragilidade estrutural cobrará um preço fatal com a era pós-quântica, para a qual os meros "montadores de componentes" são matematicamente incapazes de prover defesa. Finalmente, na fase de Implantação e Manutenção, a história do Therac-25, o recente apagão da CrowdStrike e a vulnerabilidade crônica a ataques à cadeia de suprimentos atestam empiricamente que negligenciar a homologação e a observabilidade transcende a ineficiência virtual, materializando-se em interrupções planetárias.

Desta forma, a superação deste cenário crítico exige uma reconfiguração imediata de paradigma, rompendo com o pragmatismo cego do ágil deturpado e com a leniência da cláusula "*software as is*". Diante da incidência implacável da LGPD, da responsabilização civil solidária e das sanções criminais por imperícia, exige-se a reafirmação incontestável do papel do verdadeiro Engenheiro de Software. Apenas a consolidação de uma engenharia regida por

governança técnica, *compliance* digital e legitimação jurídica consubstanciada no registro em conselhos de classe e na emissão de responsabilidade técnica será capaz de blindar o profissional contra a dinâmica corporativa do "bode expiatório" e garantir que a malha cibernética da sociedade seja arquitetada para prosperar, e não para colapsar.

REFERÊNCIAS

BAUMAN, Zygmunt. *Modernidade líquida*. Rio de Janeiro: Zahar, 2001.

BECK, Kent et al. *Manifesto para desenvolvimento ágil de software*. 2001. Disponível em: <https://agilemanifesto.org/iso/ptbr/manifesto.html>.

BEYER, Betsy et al. *Site Reliability Engineering: How Google Runs Production Systems*. Sebastopol: O'Reilly Media, 2016.

BOURDIEU, Pierre. *O poder simbólico*. Rio de Janeiro: Bertrand Brasil, 1998.

BRASIL. *Decreto-Lei nº 2.848, de 7 de dezembro de 1940*. Código Penal. Rio de Janeiro, RJ: Presidência da República, 1940.

BRASIL. *Lei nº 5.194, de 24 de dezembro de 1966*. Regula o exercício das profissões de Engenheiro, Arquiteto e Engenheiro-Agrônomo, e dá outras providências. Brasília, DF: Presidência da República, 1966.

BRASIL. *Lei nº 8.078, de 11 de setembro de 1990*. Dispõe sobre a proteção do consumidor e dá outras providências. Brasília, DF: Presidência da República, 1990.

BRASIL. *Lei nº 10.406, de 10 de janeiro de 2002*. Institui o Código Civil. Brasília, DF: Presidência da República, 2002.

BRASIL. *Lei nº 13.709, de 14 de agosto de 2018*. Lei Geral de Proteção de Dados Pessoais (LGPD). Brasília, DF: Presidência da República, 2018.

BRASSCOM. *Demanda de talentos em TIC e estratégia de formação*. São Paulo: Associação das Empresas de Tecnologia da Informação e Comunicação (TIC) e de Tecnologias Digitais, 2021.

CASTELLS, Manuel. *A sociedade em rede*. São Paulo: Paz e Terra, 2010.

CAVOUKIAN, Ann. *Privacy by Design: The 7 Foundational Principles*. Toronto: Information and Privacy Commissioner of Ontario, 2009.

COCKBURN, Alistair. *Hexagonal Architecture*. 2005. Disponível em: <https://alistair.cockburn.us/hexagonal-architecture/>.

CROWDSTRIKE. *Falcon Content Update Remediation and Guidance Hub*. Relatório de Incidente Global. Austin, TX: CrowdStrike, jul. 2024.

CUNNINGHAM, Ward. The WyCash portfolio management system. *ACM SIGPLAN OOPS Messenger*, v. 4, n. 2, p. 29-30, dez. 1992.

DONG, Yuhang et al. DevGPT: a large-scale dataset of real-world software development with ChatGPT. *ArXiv preprint arXiv:2308.00675*, 2023.

HAN, Byung-Chul. *Sociedade do cansaço*. Petrópolis: Vozes, 2015.

LEVESON, Nancy G.; TURNER, Clark S. An investigation of the Therac-25 accidents. *Computer*, v. 26, n. 7, p. 18-41, jul. 1993.

MACHLUP, Fritz. *The production and distribution of knowledge in the United States*. Princeton: Princeton University Press, 1962.

MARTIN, Robert C. *Arquitetura limpa: o guia do artesão para estrutura e design de software*. Rio de Janeiro: Alta Books, 2019.

MITRE CORPORATION. *CVE-2021-44228 (Log4j vulnerability)*. National Vulnerability Database. 2021. Disponível em: <https://nvd.nist.gov/vuln/detail/CVE-2021-44228>.

SHOR, Peter W. Algorithms for quantum computation: discrete logarithms and factoring. In: *Proceedings of the 35th Annual Symposium on Foundations of Computer Science*. IEEE, 1994. p. 124-134.

SOMMERVILLE, Ian. *Engenharia de software*. 10. ed. São Paulo: Pearson, 2019.

WIRTH, Niklaus. A plea for lean software. *Computer*, v. 28, n. 2, p. 64-68, fev. 1995.